RL hw4

110612025 魏于翔

June 23 2026

1 Problem 1

1.1 (a)

The exact gradient of the DPO loss function ($\nabla_\theta \mathcal{L}_{DPO}$) is:

$$\begin{aligned}&\nabla_\theta \mathcal{L}_{DPO}(\pi_\theta; \pi_{ref}) = \\ &-\beta E_{(x,y_w,y_l)\sim\mathcal{D}}\Big[\sigma\big(\hat{r_\theta}(x, y_l) - \hat{r_\theta}(x, y_w)\big) \\ &\cdot \big[\nabla_\theta \log \pi(y_w|x) - \nabla_\theta \log \pi(y_l|x)\big]\Big]\end{aligned} \tag{1}$$

Each part intuition behind the gradient:

- $\sigma\big(\hat{r_\theta}(x, y_l) - \hat{r_\theta}(x, y_w)$: When the model is correct, means $r_w > r_l$, so the scaling weight $w = \sigma(r_l - r_w) \to 0$, meaning that the model barely change, and if the model doesn't sure which is better, it give them medium weight. As for the model is wrong, meaning that $r_w < r_l$, it will give the model's weight $w = \sigma(r_l - r_w) \to 1$ to change the model. In conclusion, this term means that higher weight when reward estimate is wrong.

- $\nabla_\theta \log \pi(y_w|x)$: This term is the winner's policy gradient. In the policy gradient formula: $\theta \leftarrow \theta - \eta\Delta L$. We can see in the ΔL's formula, there is a negative sign outside, so the update becomes, $+\log \pi(y_w|x)$, adding the winner's probability.

- $-\nabla_\theta \log \pi(y_l|x)$: This term is the loser's policy gradient. In the policy gradient formula: $\theta \leftarrow \theta - \eta\Delta L$. We can see in the ΔL's formula, there is a negative sign outside, so the update becomes, $-\log \pi(y_l|x)$, decreasing the loser's probability.

1.2 (b)

- Question 1: In **DataCollatorForPreference** class's function **torch_call()**, it divide the prompt and the answer into prompt_chosen_ids and prompt_rejected_ids, also build the completion_mask, 0 for prompt token and 1 for response token. And then build the input_ids composed by B chosen and B rejected.
  In **_compute_loss()** function, pass the model's parameters in and get the *output.logits* and use the token_t to predict the token_(t+1) and compute each token's log-probability in per_token_logps variable, so each position has it's $\log \pi_\theta(y_t|y_{<t}, x)$. And then we imply the shift_completion_mask to clean up the prompt token's logprob, and then add it up to become the chosen_logps or rejected_logps.
  Because we design the input as B's chosen prompt and B's rejected prompt, so we just cut in the half to get the corresponding results.

- Question 2: You can choose to cut the beginning or the end of the sequence by setting the **truncation_mode** variable, if you want to truncate the end, set the variable to keep_start mode, if you want to truncate the beginning, set the variable to keep_end mode.

- Question 3: The standard DPO loss that involves $\log \sigma(x)$, this term may cause the numerical stability issue, when x is the negative number like -1000, the $\sigma(x) \to 0$, and when you want to get $\log(0) \to -\inf$, which may cause the trouble. In DPOTrainer, it handle the problem by using **logsigmoid** function in pytorch, in the function it using the mathematical identity

$\log \sigma(x) = -\log(1 + e^{-x})$ to avoid the problem we mentioned before.

- Question 4: In the original DPO loss, $h = (\log \pi_\theta(y_w|x) - \log \pi_\theta(y_l|x))$, and $L = -log\sigma(\beta h)$. However when we choose the lose type with **ipo**, TRL replace this classification-style with the IPO squared loss. First step is to convert sequence-level log-probabilities into average log-probability per completion token, because IPO' s squared loss is sensitive to response length. Without normalization, longer responses would naturally produce larger log-probability differences and therefore disproportionately larger losses. Then use the above number to calculate the average token log-probabilities. Last step is to apply the IPO objective by using $L_{IPO} = (\Delta_{IPO} - \frac{1}{2\beta})^2$. Instead of maximizing the preference margin indefinitely, IPO trains the model to match a target margin $\frac{1}{2\beta}$.

Figure 1:

- Question 5: TRL build a frozen reference policy π_{ref}. In normal case: we copy the policy and froze it so it can't change. In PEFT case: Shut down the adapter, use base model as reference.

Figure 2:

2 Problem 2

(a)

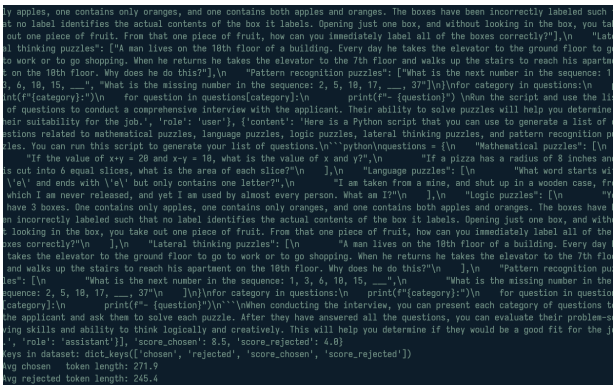

Figure 3:

Brief Summary:

	chosen	rejected
Average token length	271.9	245.4

(b) I have been trying to run the baseline DPO training, but I consistently encounter a CUDA Out of Memory (OOM) error at the very first training step (during self.ref_model(...) in DPOTrainer.compute_loss). My environment is: Python 3.10 PyTorch 2.12 Transformers 5.11 TRL 1.6.0 Weights & Biases 0.27 GPU: NVIDIA GeForce RTX 4050 Laptop GPU (6GB VRAM) I have tried the following modifications: Set torch_dtype=torch.bfloat16 when loading the model.

- Enabled gradient_checkpointing.
- Set model.config.use_cache = False.
- Set per_device_train_batch_size = 1.
- Set per_device_eval_batch_size = 1.
- Used gradient_accumulation_steps = 8.
- Reduced max_length from 1024 to 512.

Confirmed that no other GPU-intensive processes were running (nvidia-smi showed almost all VRAM was available before training). Verified that my package versions match the homework requirements. Despite these changes, the training still fails with a CUDA OOM error before completing the first training step. The error occurs when the reference model performs its forward pass inside DPO training. Based on these results, I suspect that the 6GB VRAM on my GPU may not be sufficient to run the baseline DPO training.